

Object-oriented Programming

Contents

1	Java	4
1.1	Java as an object-oriented language	4
1.2	Java virtual machine	4
1.3	Classes overview	4
1.4	Objects overview	5
1.5	Overview of OOP principles	5
1.5.1	Encapsulation overview	5
1.5.2	Inheritance overview	5
1.5.3	Polymorphism overview	6
1.5.4	Abstraction overview	6
1.6	Java basics	7
1.6.1	Hello world	7
1.6.2	Data types	7
1.6.3	Declaration and assignment	8
1.6.4	Semi-colons	8
1.6.5	Branching	8
1.6.6	Loops	9
1.7	Classes	9
1.7.1	<code>this</code>	10
1.7.2	Constructors	10
1.8	Creating an object	11
1.9	Object and reference	11
1.9.1	Reference copying	12
1.10	Encapsulation	12
1.10.1	Access modifiers	12
1.11	<code>static</code>	13
1.11.1	<code>static final</code>	14
1.11.2	<code>static</code> methods	14
1.12	Inheritance	15
1.12.1	Method overriding	15
1.12.2	<code>super</code>	16
1.12.3	<code>protected</code>	17
1.13	Abstract classes	17
1.14	Interfaces	18
1.15	UML class diagrams	19
1.15.1	UML arrows	19

1.15.2	Access modifiers in UML	21
1.16	Abstraction	22
1.16.1	Abstraction vs encapsulation	22
1.16.2	Benefits of abstraction	22
1.16.3	Implementing abstraction	23
1.16.4	static interface methods	23
1.16.5	default interface methods	23
1.16.6	Interfaces as a type	23
1.16.7	Multi-inheritance	24
1.16.8	When to use abstract classes or interfaces	24
1.17	Polymorphism	25
1.17.1	Method overloading (compile-time polymorphism)	25
1.17.2	Method overriding (run-time polymorphism)	25
1.17.3	Upcasting	25
1.17.4	Downcasting	27
1.17.5	instanceof	27
1.18	Generics	28
1.18.1	Generic class	28
1.18.2	Generic method	29
1.18.3	Primitive types in generics	30
1.18.4	Type erasure	30
1.18.5	Bounded generics	30
1.19	Collections	30
1.19.1	Arrays vs collections	31
1.19.2	Java collections categories	32
1.19.3	LinkedList	32
1.19.4	ArrayList vs LinkedList	32
1.19.5	Looping over lists	32
1.19.6	Iterators	33
1.19.7	HashMap	34
1.19.8	HashSet	34
1.20	Stack and heap	35
1.20.1	Stack	35
1.20.2	Heap	35
1.20.3	Objects and references in collections	35
1.20.4	Copy constructor	36
1.21	Design patterns	37
1.21.1	The Singleton pattern	37
1.22	The Abstract Factory Pattern	39
1.22.1	Key idea	39
1.22.2	Example structure (conceptual)	40
1.22.3	UML	40
1.23	The Factory Method Pattern	40
1.23.1	Key idea	40
1.23.2	Characteristics	40
1.23.3	Example (simple factory method style)	41
1.23.4	Key difference	41
1.23.5	Abstract Factory Pattern	41

1.24	The Observer Pattern	42
1.24.1	Key idea	42
1.24.2	Common uses	42
1.24.3	Roles	42
1.24.4	UML	43

1 Java

Java is a high-level, object-oriented programming language.

It follows the principle of Write Once, Run Anywhere (WORA), meaning that Java programs are compiled into bytecode that can run on any system with a compatible Java virtual machine (JVM). It is commonly used in web applications, mobile development, and enterprise software.

1.1 Java as an object-oriented language

Java is based on four key object-oriented programming (OOP) principles:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Everything in Java is an **object**:

- Objects contain attributes (data/state) and methods (behaviour)

A **class** is a blueprint or template that defines the fields and methods that its objects will have.

- This promotes modular, reusable, and maintainable code

1.2 Java virtual machine

The Java virtual machine (JVM) is a virtual machine that executes Java bytecode. It:

- provides a platform-independent runtime environment
- translates bytecode into machine code for the host system
- enables Java applications to run on different operating systems

1.3 Classes overview

A class is a template for creating objects:

- it defines state (variables) and behavior (methods)
- a class does not occupy memory until an object is instantiated

An example of a class is given below:

```
class Car {  
    // variables  
    String brand;  
    int speed;  
  
    // methods  
    void accelerate() {  
        speed += 10;  
    }  
}
```

1.4 Objects overview

An object is an instance of a class:

- it represents a specific realisation of the class
- it has state (attributes) and behaviour (methods)
- objects are stored in heap memory at runtime

An example of creating an object from the `Car` class is given below:

```
public class TestCar {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.brand = "Toyota";  
        myCar.accelerate();  
        System.out.println("Brand: " + myCar.brand + " Speed: " + myCar.speed);  
    }  
}
```

1.5 Overview of OOP principles

1.5.1 Encapsulation overview

Encapsulation means hiding internal details and exposing only what is necessary:

- private variables cannot be accessed directly from outside the class and are accessed via public methods (getters and setters)

```
class Person {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

1.5.2 Inheritance overview

Inheritance allows a new class to inherit from an existing class:

- this promotes code reusability and hierarchy

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```

class Dog extends Animal {
    // the Dog class inherits any attributes
    // and methods defined in the Animal class
    void makeSound() {
        System.out.println("Dog barks");
    }
}

```

1.5.3 Polymorphism overview

Polymorphism allows a method to take different forms:

- method overriding is when a subclass provides a different implementation of a method from the parent class
- method overloading is when multiple methods have the same method name but different parameter lists (signature), possibly with different return behaviour

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

```

```

class Dog extends Animal {
    void makeSound() {
        // the Dog class overrides the makeSound()
        // function that it inherited
        System.out.println("Dog barks");
    }
}

```

1.5.4 Abstraction overview

Abstraction hides implementation details and only shows relevant functionality:

- defines a template that enforces certain methods while hiding implementation details
- achieved using abstract classes or interfaces

```

abstract class Animal {
    // No implementation
    abstract void makeSound();
}

class Cat extends Animal {
    void makeSound() {
        System.out.println("Meow");
    }
}

```

1.6 Java basics

1.6.1 Hello world

To create a basic Java program:

- save the code in a `.java` file
- the file may contain a class (only one public class allowed per file)
- if a class is declared `public`, the file name must match the class name
- the program must contain a `main` method
 - defined as `public static void main(String args[])`
- the JVM starts program execution from the `main` method

The implementation of ‘Hello World’ is given below:

```
[1]: // MyMain.java  
class MyMain {  
    public static void main(String args[]){  
        System.out.println("Hello World!");  
    }  
}  
  
// Normally, Java programs run automatically  
// by the JVM starting from main.  
// In Jupyter notebooks,  
// the main method must be called manually:  
MyMain.main(new String[] {});
```

Hello World!

1.6.2 Data types

Data types are split into primitive and non-primitive types.

Java has 8 primitive data types:

- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `char`
- `boolean`

Non-primitive types include:

- `String`
- `Arrays`
- `Classes`

1.6.3 Declaration and assignment

Every variable in Java must have a declared data type before it is used.

The syntax for declaring a variable is:

```
type variableName;
```

e.g.

```
int x;  
String y;
```

Values can be assigned using =:

```
x = 5;  
y = "ABC";
```

Declaration and assignment can be done in a single statement:

```
int x = 5;
```

1.6.4 Semi-colons

Most executable statements in Java must end in a semicolon (;).

e.g.,

```
int x = 5;  
System.out.println(x);
```

However, semicolons are not used after:

- class declarations
- method declarations
- control structures like `if`, `for`, and `while`

1.6.5 Branching

Java supports conditional branching using `if`, `else if`, and `else` statements.

Branching allows a program to make decisions and execute different code depending on conditions.

The syntax is:

```
if (condition) {  
    // code  
}  
else if (condition) {  
    // code  
}  
else {  
    // code  
}
```

The `else if` and `else` clauses are optional.

An example use is:

```
[2]: int a = 10;

if (a < 0) {
    System.out.println("Negative");
}
else {
    System.out.println("Positive");
}
```

Positive

1.6.6 Loops

Loops allow repeated execution of a block of code.

for loops for loops are used when the number of iterations is known:

```
for (int i = 0; i < 5; i++) {
    System.out.println("i=" + i);
}
```

- `continue` skips the rest of the current iteration and moves to the next iteration.
- `break` can be used to exit the loop.

while loops while loops repeat execution while a condition is true:

```
int j = 0;
while (j < 10) {
    System.out.println("j=" + j);
    j += 1;
}
```

- `break` can be used to exit the loop.

1.7 Classes

A class is typically made up of three main components:

- Fields (instance variables) - store the object's state (data)
- Methods - define the object's behaviour
- Constructors - initialise new objects

An example is given below:

```
class Person {
    // fields
    private String name;
    private int age;

    // constructor
    public Person(String name, int age) {
```

```

        this.name = name;
        this.age = age;
    }

    // class methods
    // getter for name
    public String getName() { return name; }

    // setter for name
    public void setName(String name) {
        this.name = name;
    }

    // getter for age
    public int getAge() { return age; }

    // setter for age
    public void setAge(Int age) {
        this.age = age;
    }

    // method to display stored information
    public void displayInfo() {
        System.out.println("Name: " + name +
                           ", Age: " + age);
    }
}

```

In this example:

- the fields are **private**
 - they cannot be accessed directly from outside the class
- the constructor and methods are **public**
 - they can be called from other classes

1.7.1 this

The **this** keyword is used to refer to the current object in a method or constructor.

It is commonly used to eliminate confusion between class attributes and parameters with the same name.

1.7.2 Constructors

A constructor:

- must have the same name as the class
- must be **public**
- has no return type (not even **void**)
- is called automatically when an object is created with **new**

- initialises the object's fields

Note that a constructor does not have to be explicitly provided. If one is not provided, one will be created automatically by the compiler.

Multiple constructors Constructors can be overloaded, meaning a class can define multiple constructors with different parameter lists.

This is an example of method overloading, a form of polymorphism.

The constructor that is called depends on the arguments provided when the object is created.

For example, in addition to

```
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}
```

another constructor can be defined:

```
public Person() {
    this.name = "No Name";
    this.age = 0;
}
```

This allows a `Person` object to be created without providing any arguments. In this case, the fields are initialised with default values chosen by the constructor.

The same applies to other (non-constructor) methods:

- multiple implementations of the same method can be defined with different parameter lists, and the appropriate one is called depending on the arguments provided

1.8 Creating an object

An object is created using the `new` keyword, which calls a constructor.

```
ClassName objectName = new ClassName(arguments);
```

For example, for the `Person` class, objects can be created and initialised with:

```
Person myPerson = new Person("Greg", 42);
// calls Person(String name, int age)
```

```
Person myUnnamedPerson = new Person();
// calls Person()
```

1.9 Object and reference

When the following statement is executed:

```
ClassName objectName = new ClassName(arguments);
```

the following happens:

- `ClassName objectName` declares a reference variable named `objectName` that references a `ClassName` object
- `new ClassName(arguments)` creates a new object in memory and calls the constructor
- the reference to the newly created object is assigned to `objectName`

Together, this statement creates an object and stores a reference to it in the variable.

1.9.1 Reference copying

A reference variable can be copied so that two variables point to the same object.

```
Person myPerson1 = new Person("Greg", 42);
Person myPerson2 = myPerson1;
```

Now both variables refer to the same object in memory. Therefore:

- if the object is modified through one reference, the change is visible through the other

1.10 Encapsulation

Encapsulation in OOP is the bundling of data (fields) and methods that operate on that data into a single unit (a class), while restricting direct access to some of the object's internal state.

The benefits of encapsulation are that it:

- protects data from accidental/erroneous modification
- enhances data hiding and security
- promotes modular and maintainable code

Java achieves this using access modifiers, mainly:

- `private`, and
- `public`

1.10.1 Access modifiers

`private` and `public` are access modifiers that control how much of a class is visible to other parts of the program, allowing only necessary access.

They are used to ensure other classes access only the minimum information needed to perform their tasks.

`private` makes a variable/method accessible only within the same class.

`public` makes a variable/method accessible from anywhere in the program.

For a `private` field, `public` methods (called getters and setters) can be used to access and update the field, and can include validation to ensure that updates are valid.

```
class Person {
    // both fields are private
    private String name;
    private int age;

    // setter for name
```

```

public void setName(String newName) {
    this.name = newName;
}

// getter for name
public String getName() {
    return name;
}

// setter for age (with validation)
public void setAge(int newAge) {
    if (newAge > 0) {
        this.age = newAge;
    } else {
        System.out.println("Invalid age!");
    }
}

// getter for age
public int getAge() {
    return age;
}
}

```

1.11 static

The **static** keyword in Java is used to create class-level members that belong to the class rather than an instance of the class:

- instance members belong to objects (each object gets its own copy)
- static members belong to the class itself (shared across all instances of the class)

```

class ExampleClass {
    static int staticVar = 10;
    // shared across all objects

    int instanceVar = 20;
    // separate for each object
}

```

Only one copy of the **static** variable exists, regardless of how many objects are created. Therefore, updating the value for one object updates it for all objects of that class (since the variable is shared).

Since **static** variables belong to the class, they can be accessed through an instance or directly through the class. However, the recommended way is using the class name.

For example, `staticVar` can be accessed using

```
ExampleClass.staticVar
```

without needing to create any instances of `ExampleClass`.

```
[3]: class Counter {
    static int staticCounter = 0;
    int instanceCounter = 0;
}

Counter counter1 = new Counter();
counter1.staticCounter++;
counter1.instanceCounter++;

Counter counter2 = new Counter();
counter2.staticCounter++;
counter2.instanceCounter++;

// staticCounter is shared
// Each object has their own instanceCounter
System.out.println(
    "counter1.staticCounter = " + counter1.staticCounter +
    "\ncounter1.instanceCounter = " + counter1.instanceCounter +
    "\ncounter2.staticCounter = " + counter2.staticCounter +
    "\ncounter2.instanceCounter = " + counter2.instanceCounter
);
```

```
counter1.staticCounter = 2
counter1.instanceCounter = 1
counter2.staticCounter = 2
counter2.instanceCounter = 1
```

1.11.1 static final

The `static final` combination is used for constants. These are variables that cannot be changed after initialisation.

```
class Config {
    static final double PI = 3.14159;
}
```

By convention, constants are written in uppercase.

1.11.2 static methods

A `static` method belongs to the class, not to objects.

It cannot access instance (non-static) variables/methods directly:

- this is because static methods exist without any specific object instance
- a static method can therefore run without any object being guaranteed to exist

It can however use instance variables by passing an object (or values) as arguments:

```
class Person {
    int age;
```

```

static void printAgeFromObj(Person p) {
    System.out.println(p.age);
}

static void printAgeFromVal(int pAge) {
    System.out.println(pAge);
}
}

```

1.12 Inheritance

Inheritance is an OOP concept that allows a subclass (child class) to be derived from a superclass (parent class), inheriting its fields and methods and optionally adding new features or overriding existing behaviour.

Inheritance has many benefits:

- code reusability
 - shared behaviour is written once in the parent class
 - subclasses reuse it automatically
- easier maintenance
 - if the parent class is updated, all subclasses automatically get the update
- hierarchy modelling
 - inheritance helps model real-world relationships
 - * e.g., `Dog` is a subclass of `Animal`
 - this makes programs more natural and organised
- extensability
 - shared behaviour can remain in the parent class
 - specialised behaviour can be added in the subclass

In Java, the `extends` keyword is used to create a subclass:

```

class SuperClass {
    // Parent properties and methods
}

class DerivedClass extends SuperClass {
    // Additional properties and methods
}

```

The subclass inherits all non-`private` members of the superclass.

1.12.1 Method overriding

Method overriding occurs when a subclass provides a new implementation of a method that is already defined in its superclass.

For a method to be overridden, the subclass method must have the same:

- method name
- parameter list

- return type

In Java, the `@Override` annotation is used to indicate that a method is intended to override a superclass method:

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override void makeSound() {
        System.out.println("Dog barks");
    }
}
```

Note that using `@Override` is not strictly necessary, but is good practice.

If `@Override` were removed from the example above, the program would still behave exactly the same.

`@Override` should be used because:

- it makes it clear to readers that the method is overriding a superclass method
- the compiler checks that the method actually overrides a valid method
- it helps catch mistakes such as:
 - typos in the method name
 - incorrect parameter lists
 - incorrect return types

Without `@Override`, if the method signature does not match the superclass method, Java treats it as a new overloaded method rather than an overridden one.

1.12.2 super

The `super` keyword is used to refer to the immediate superclass (parent class) of the current object.

It is commonly used to:

- call a superclass constructor using `super(arguments)`
- call a superclass method using `super.methodName(arguments)`
- access a superclass field using `super.fieldName`

Calling a superclass constructor A subclass constructor must call a superclass constructor.:

- if the constructor does not explicitly call `super(...)`, Java automatically inserts a call to `super()` as the first statement
- therefore, if the superclass does not have a no-argument constructor, the subclass must explicitly call one of the superclass constructors
- if used, `super(...)` must be the first statement in the subclass constructor.

```
class Animal {
    String name;
```

```

    Animal(String name) {
        this.name = name;
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);    // calls Animal(String name)
    }
}

```

Calling a superclass method `super.methodName(arguments)` calls the superclass version of a method, even if it has been overridden in the subclass.

```

[4]: class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        super.makeSound(); // calls Animal.makeSound()
        System.out.println("Bark");
    }
}

Dog myDog = new Dog();
myDog.makeSound();

```

```

Animal sound
Bark

```

1.12.3 protected

The **protected** access modifier allows a variable or method to be accessed in a more flexible way than private, but still restricts full public access.

A protected member is accessible:

- within the same class
- within the same package
- in subclasses

It is not accessible from unrelated classes outside the package.

1.13 Abstract classes

Abstract classes are declared using the **abstract** keyword and are used as base classes that cannot be instantiated directly (they can only be used as superclasses).

They may contain:

- abstract methods (methods without a body)
- concrete methods (methods with a body)
- fields (variables)

Abstract methods:

- are also declared using the **abstract** keyword
- do not have an implementation
- must be implemented by concrete (non-abstract) subclasses

```
abstract class Animal {
    abstract void makeSound();
}
class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }
}
```

1.14 Interfaces

An interface in Java defines a contract that classes must follow. It specifies what a class must do, but not how it does it.

- Interfaces are declared using the **interface** keyword.
- Before Java 8, interfaces contained only abstract methods. After Java 8, non-abstract methods are also allowed.
- They define behaviours that a class must implement, and support multiple inheritance (meaning a class can inherit multiple interfaces)
 - this is a key difference between interfaces and abstract classes; a class cannot inherit multiple superclasses

A class uses the **implements** keyword to follow an interface.

- it must provide implementations for all abstract methods

```
interface ElectricVehicle {
    void chargeBattery();
}

interface Vehicle {
    void start();
    void stop();
}

class ElectricCar implements Vehicle, ElectricVehicle {
    public void start() {
        System.out.println("Electric car starts silently.");
    }
    public void stop() {
```

```

        System.out.println("Electric car is stopping.");
    }
    public void chargeBattery() {
        System.out.println("Charging battery...");
    }
}

```

1.15 UML class diagrams

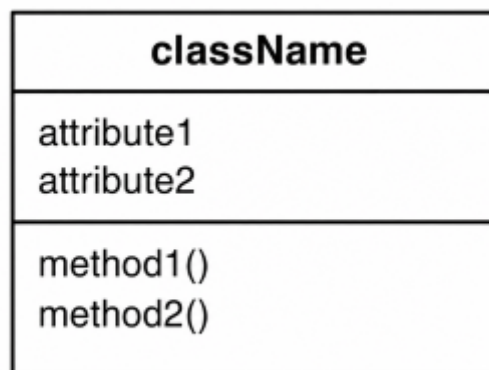
Unified Modelling Language (UML) can be used to draw class diagrams, which show the fields, methods and relationships of classes.

The representation of a class in UML has three main sections:

- the class name
- the fields
- the methods

The class name is the only required section.

Static members (variables and methods) are shown by underlining their names.



1.15.1 UML arrows

Different arrows have different meanings.



Directed association A directed association arrow shows a directed relationship - one class uses or depends on another, but not necessarily the other way around.

- one class references or uses another class



Composition A composition relationship is a strong form of association.

It indicates a class owns another class:

- the part class cannot exist independently of the whole class
- the part is created and destroyed with the whole
- strong 'has-a' relationship
- e.g. a **Car** class has an **Engine**



Aggregation Aggregation is a weaker form of association than composition.

It indicates a class contains/collects another class:

- the part class can exist independently of the whole class
- the part is not destroyed when the whole is destroyed
- e.g. a **Professor** class has multiple **Course** objects - the courses can still exist even if the professor stops teaching them



Inheritance An inheritance arrow shows that a subclass (child class) inherits from a superclass (parent class).

It represents an 'is-a' relationship, where the subclass inherits the fields and methods of the superclass.

The arrow points from the child → parent.



Dependency A dependency relationship in UML shows that one class uses another class temporarily, without owning it as a field:

- a dependency means one class relies on another class to perform some function
- the class being used is not stored as an attribute (no long-term relationship)
- the arrow points from the using class to the used class

```
class Printer {  
    void print(String text) {  
        System.out.println(text);  
    }  
}  
  
class Report {  
    void generate() {  
        Printer printer = new Printer();  
        printer.print("Report generated");  
    }  
}
```

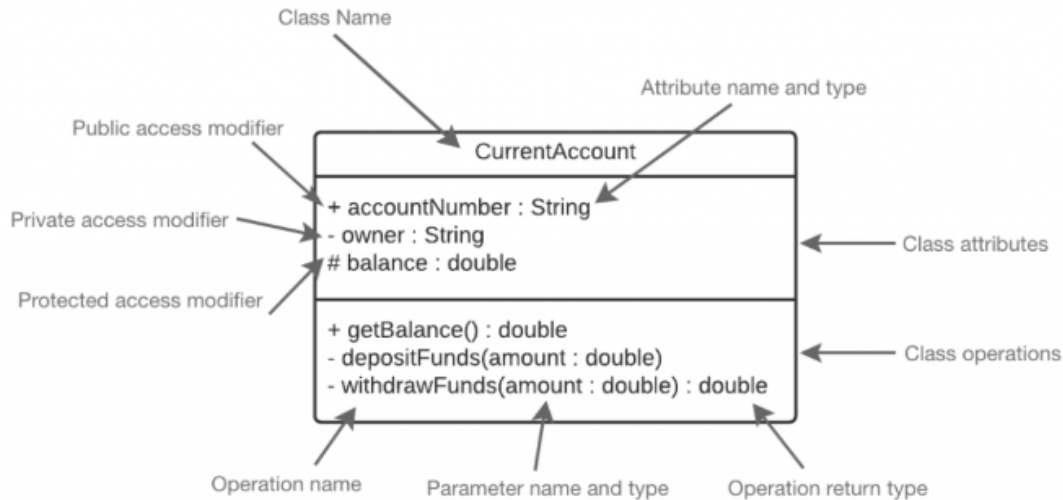
Here, `Printer` is a dependency of `Report`, as `Printer` is not stored as a field in `Report`:

- it is created and used only inside a method
- the relationship is temporary ('uses-a') rather than long-term ownership

1.15.2 Access modifiers in UML

- `public` members are prefixed by the symbol `+`
- `private` members are prefixed by the symbol `-`
- `protected` members are prefixed by the symbol `#`

An annotated example of a UML class is given below.



1.16 Abstraction

Abstraction in Java allows a superclass or interface to define the structure (method signatures) while leaving the actual implementation to subclasses:

- the superclass implements only the high-level design (e.g. method signatures for abstract methods) without knowing the specific logic, without implementing the specific logic, which becomes the responsibility of the subclass
- abstract classes and interfaces specify what operations are available, without defining how they are carried out

This ensures that different objects can share a common interface (behaviour, standard) without revealing the implementation details.

1.16.1 Abstraction vs encapsulation

Abstraction:

- Defines what a class should do but not how
- Hides implementation details by providing method signatures without actual implementations in abstract classes or interfaces.
 - the child class must implement the details
- Focuses on reducing complexity by showing only essential behaviour

Encapsulation: - Hides internal data and restricts access to it using access modifiers (**private**, **protected**, **public**). - Ensures that only necessary parts of an object are accessible from outside. - Focuses on protecting and controlling access to an object's state

1.16.2 Benefits of abstraction

Using abstraction:

- reduces complexity by hiding implementation details and exposing only essential behaviour
- enhances code reusability by allowing different classes to share the same interface or abstract structure

- improves maintainability because changes to implementation do not affect code that depends on the abstraction (as long as the interface remains the same)

1.16.3 Implementing abstraction

In Java, abstraction is implemented using:

- **abstract classes**
- **interfaces**

Before Java 8, abstract classes and interfaces were quite different, as interfaces could only contain method declarations and constants. Since Java 8, interfaces can also include concrete implementations through **default** and **static** methods, making them more similar to abstract classes.

However, important differences remain:

- abstract classes can contain instance variables, constructors, and methods with any access modifier
- interfaces cannot have instance variables or constructors, and their methods are public by default
- a class can extend only one abstract class, but it can implement multiple interfaces

1.16.4 static interface methods

Interfaces can contain **static** methods:

- these are called using the interface name, not through an object of an implementing class
- they are not inherited by **implementing** classes
- they are typically used for utility methods related to the interface

1.16.5 default interface methods

Interfaces may also contain **default** methods:

- these are declared using the **default** keyword and include a method body
- if no keyword is specified, the method is implicitly **public abstract**

default methods:

- are inherited by implementing classes
- can be overridden by implementing classes
- if two or more interfaces provide a default implementation for the same method, the implementing class must provide its own implementation to avoid ambiguity

1.16.6 Interfaces as a type

Interfaces can be used as a type for variables, method parameters, and return types.

This allows code to work with any object that implements the interface, rather than depending on a specific class.

For example, if a method parameter has an interface type, the method can accept any object whose class implements that interface.

```
[5]: interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Woof");
    }
}

static void makeSound(Animal a) {
    a.sound();
    // any variable that implements the
    // Animal interface must implement
    // the sound() function
}

Dog myDog = new Dog();
myDog.sound();
```

Woof

1.16.7 Multi-inheritance

Multiple inheritance means that a class inherits from two or more parent classes.

Java does not allow multiple inheritance of classes:

- Java supports only single inheritance for classes
- a class can extend only one superclass
- however, a class can implement multiple interfaces

This restriction avoids the diamond problem:

- if a class inherits from two superclasses that both provide different implementations of the same method, it becomes ambiguous which implementation should be inherited

Interfaces help avoid this issue:

- traditionally, interfaces contained only method declarations, so there was no ambiguity because the implementing class provided the implementation
 - since Java 8, interfaces can include **default** methods
- if two interfaces provide the same default method, the implementing class must override the method to resolve the conflict explicitly

1.16.8 When to use abstract classes or interfaces

Abstract classes should be used when:

- code (variables and methods) needs to be shared among closely related classes
 - especially when sharing state (instance variables), since interfaces cannot have instance fields

- there is partially implemented behaviour that subclasses will extend or customise
- classes share a strong ‘is-a’ relationship and a common base implementation is useful

Interfaces should be used when:

- a contract needs to be enforced between potentially unrelated classes
- multiple inheritance of type is required
- you want to define behaviour without relying on shared state
- default behaviour is needed via **default** methods, but without shared instance variables

1.17 Polymorphism

Polymorphism means ‘many forms’.

In OOP, it allows a single “thing” (object or method) to behave in different ways depending on context (different classes or implementations).

It is achieved through:

- Method overloading (compile-time polymorphism)
- Method overriding (run-time polymorphism)
- Upcasting and downcasting (type conversion in inheritance)

Polymorphism improves reusability, flexibility and maintainability.

1.17.1 Method overloading (compile-time polymorphism)

Method overloading is when multiple methods have the same name but different parameter lists. It happens within the same class.

The compiler determines which method is called at compile-time based on the method signature.

Method overloading was covered earlier, with an example, in the [Classes](#) section.

1.17.2 Method overriding (run-time polymorphism)

Method overriding allows a subclass to provide a specific implementation of a method already defined in its superclass.

The method that is executed is determined at run-time based on the actual object type (not the reference type).

Method overriding was covered earlier, with an example, in the [Inheritance](#) section.

1.17.3 Upcasting

Upcasting is when a subclass object is assigned to a superclass reference.

Only methods defined in the superclass are accessible. However any overridden methods will still use the subclass implementation (run-time polymorphism).

```
[6]: class Animal {
      void makeSound() { System.out.println("Animal sound."); }
    }
```

```

class Dog extends Animal {
    void makeSound() { System.out.println("Bark!"); }
    void walk() { System.out.println("Walking..."); }
}

Animal myPet = new Dog(); // upcasting
// a Dog object is referenced as an Animal

myPet.makeSound();
// calls Dog's overridden method at runtime

// myPet.walk();
// not accessible (not defined in Animal)

```

Bark!

Upcasting in method parameters Upcasting allows a method that takes a superclass parameter to accept objects of any subclass.

The subclass object is implicitly upcast to the superclass type when passed as an argument.

```

[7]: Dog myDog = new Dog();

void makeSound(Animal a) {
    // expects object with class Animal
    a.makeSound();
}

makeSound(myDog);

```

Bark!

Upcasting in arrays and collections Upcasting also allows arrays and collections to store objects of different subclasses, as long as they share a common superclass.

```

[8]: class Cat extends Animal {
    void makeSound() { System.out.println("Meow!"); }
}

class Cow extends Animal {
    void makeSound() { System.out.println("Moo!"); }
}

Cat myCat = new Cat();
Cow myCow = new Cow();

Animal[] zoo = {myDog, myCat, myCow};
// array of Animal objects

```

```
// each object is implicitly upcast to Animal

for (int i = 0; i < zoo.length; i++){
    zoo[i].makeSound();
}
```

Bark!
Meow!
Moo!

1.17.4 Downcasting

Downcasting is when a superclass reference is converted (typecasted) back to a subclass reference:

- it must be performed explicitly using a cast
- it allows subclass-specific methods and fields to be accessed
- it is only safe if the object is actually an instance of the subclass

```
[9]: Animal myPet = new Dog(); // upcasting
Dog myDog = (Dog) myPet; // downcasting

myDog.walk();
// can use methods defined in Dog
// but not in Animal
```

Walking..

Downcasting only works when for reverting an object back to a class it was originally upcast from.

1.17.5 instanceof

The `instanceof` is a binary operator that tests whether an object is an instance of a given class or interface.

It returns `true` if the object is of the specified type (or a subtype), and `false` otherwise.

```
[10]: Animal myPet = new Dog();

System.out.println(myPet instanceof Animal);
System.out.println(myPet instanceof Dog);
System.out.println(myPet instanceof Cat);
System.out.println(myPet instanceof Cow);
```

```
true
true
false
false
```

`instanceof` can be used to check whether a downcast is safe before performing it.

```
[11]: void takeForWalk(Animal a) {
        if (a instanceof Dog) {
            Dog myDog = (Dog) a;
            myDog.walk();
        }
        else {
            System.out.println("Cannot walk");
        }
    }

    Animal myPet = new Dog();
    System.out.println("myPet:");
    takeForWalk(myPet);

    Animal myPet2 = new Cow();
    System.out.println("myPet2:");
    takeForWalk(myPet2);
```

```
myPet:
Walking...
myPet2:
Cannot walk
```

If the compiler can determine that an `instanceof` test can never be `true`, it reports a compile-time error.

This happens when the two types are unrelated, so no object could possibly be an instance of both.

For example,

```
Cat myCat = new Cat();
System.out.println(myCat instanceof Dog);
// produces compile time error
// Cat and Dog classes are unrelated
// an object that can be a Cat can never be a Dog
```

1.18 Generics

Generics enable types to be used as parameters. This makes it possible to write code for different data/object types using a single class or method while maintaining type safety.

The angle brackets `<T>` is used to denote a generic type parameters. Multiple generic types can be used, such as `<T, U, V, ...>`.

1.18.1 Generic class

The example below is for a class that works with different data types using generics.

```
[12]: class Box<T> {
        T value; // T is a placeholder for any data type
```

```

    void setValue(T value) {
        this.value = value;
    }

    T getValue() {
        return this.value;
    }
}

// Create a Box to hold a String
Box<String> stringBox = new Box<>();
stringBox.setValue("Hello");
System.out.println("Value: " + stringBox.getValue());

// Create a Box to hold an Integer
Box<Integer> intBox = new Box<>();
intBox.setValue(50);
System.out.println("Value: " + intBox.getValue());

```

Value: Hello

Value: 50

1.18.2 Generic method

It is also possible to create methods that work with any data type using generics:

```

[13]: // Generic method: works with any type T
public static <T> void printArray(T[] arr) {
    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
}

// Array of Strings
String[] names = {"Eric", "Robert"};

// Array of Integers
Integer[] numbers = {1, 2, 3};

// Call the generic method with both arrays
printArray(names);
printArray(numbers);

```

Eric

Robert

1

2

3

1.18.3 Primitive types in generics

Primitive types (`int`, `float` etc.) cannot be type parameters.

- instead, wrapper classes (`Integer`, `Float` etc.) must be used.

1.18.4 Type erasure

During compilation, type parameters are replaced with the `Object` type (for unbounded generics).

- `Object` is the root superclass of all classes
- it contains contains basic methods that all classes (e.g. `toString()`)

```
class Box<T> {  
    T value;  
}
```

becomes

```
class Box {  
    Object value;  
}
```

Therefore, in a method or class that uses unbounded generics, only operations available to `Object` are allowed.

1.18.5 Bounded generics

Type parameters can be restricted to only allow types that extend a particular class or implement a particular interface.

```
<T extends Number>
```

This means `T` can only be `Number` or any subclass of `Number` (e.g. `Integer`, `Double`, etc.).

Now, the type parameter is not replaced with `Object`.

Instead, it is replaced with the upper bound (`Number` in this case).

```
class Box<T extends Number> {  
    T value;  
}
```

becomes

```
class Box {  
    Number value;  
}
```

Therefore, methods defined in `Number` are allowed.

1.19 Collections

The Java Collections Framework (JCF) is a unified architecture for handling and manipulating groups of objects efficiently.

It provides a set of interfaces and classes for storing, retrieving, and processing data dynamically.

It offers many benefits:

- reusability - provides standardised, well-tested data structures and algorithms
- performance - includes optimised implementations for operations like sorting and searching
- flexibility - supports dynamic resizing (unlike arrays, which are fixed size)
- thread safety - some collections support concurrent access
- reduced boilerplate code - removes the need to implement common data structures manually

Items in the JCF need to be imported before they can be used. They are imported from `java.util`.

```
import java.util.*;
```

This imports all classes in the `java.util` package.

Alternatively, specific classes can be imported. For example:

```
import java.util.ArrayList;
```

This imports only the `ArrayList` class.

1.19.1 Arrays vs collections

Arrays In Java, an array can be declared using:

```
typeName[] arrayName = new typeName[arrayLength];
```

For example,

```
int[] nums = new int[5];
```

creates an array of `ints` called `nums`, with length 5.

Arrays:

- have fixed size (cannot grow or shrink after creation)
- require manual handling for insertions and deletions (shifting elements)
- have no built-in methods for sorting, searching, or managing data

Collections A collection can be declared using:

```
List<typeParameter> collectionName = new ArrayList<>();
```

For example,

```
List<Integer> nums = new ArrayList<>();
```

creates a dynamically sized list of integers called `nums`.

An element can be accessed by index using

```
collectionName.get(index)
```

Collections:

- support dynamic resizing (can grow and shrink at runtime)
- provide built-in methods such as `add()`, `remove()`, `contains()`, `sort()`
- use generics to enforce type safety (e.g. `List<Integer>`)
- can store objects (primitive types are auto-boxed, e.g. `int` → `Integer`)

1.19.2 Java collections categories

The Java Collections Framework is divided into four main categories: 1. **List**: an ordered collection that allows duplicates 2. **Set**: a collection that does not allow duplicate elements 3. **Map**: stores key-value pairs with unique keys 4. **Queue**: first-in-first-out FIFO structure

1.19.3 LinkedList

In addition to `ArrayList`, the JCF provides a `LinkedList`:

- elements are stored in nodes (containers)
- a `LinkedList` is a doubly-linked list:
 - each node contains a reference to the next node and the previous node
 - the list maintains references to the first (head) and last (tail) nodes

1.19.4 ArrayList vs LinkedList

Each has relative advantages:

- it is faster to access random elements in an `ArrayList`
 - elements can be directly accessed by index in constant time
 - in a `LinkedList`, nodes must be traversed sequentially from the head (or tail) until the target is reached
- it is quicker to add or remove elements from a `LinkedList`
 - only the references of the adjacent nodes need to be updated
 - in an `ArrayList`, elements may need to be shifted in memory to maintain contiguous storage

1.19.5 Looping over lists

Arrays, `ArrayLists` and `LinkedLists` can be iterated using a `for` loop.

This can be done by iterating over each index until the size of the list is reached:

```
[14]: List<String> names = new ArrayList<>();

names.add("Robert");
names.add("Allison");
names.add("Eric");

for (int i = 0; i < names.size(); i++) {
    System.out.println(names.get(i));
}
```

```
Robert
Allison
Eric
```

Enhanced for loop (for-each loop) The enhanced for loop can also be used, which directly accesses each element in the list:

```
for (typeName element : listName) {
    // code
}
```

For example:

```
[15]: for (String name : names) {
        System.out.println(name);
    }
```

```
Robert
Allison
Eric
```

1.19.6 Iterators

The `Iterator` class from the JCF can also be used to loop over a collection such as a list.

An iterator is created using:

```
Iterator<typeName> iteratorName = listName.iterator();
```

Then, a while loop can be used with the `hasNext()` and `next()` methods to traverse the collection.

- `hasNext()` returns `true` if there is another element, otherwise `false`
- `next()` returns the next element in the iteration and advances the iterator

For example,

```
[16]: Iterator<String> iterNames = names.iterator();

while (iterNames.hasNext()) {
    System.out.println(iterNames.next());
}
```

```
Robert
Allison
Eric
```

Iterators are useful because:

- they allow removal of elements while traversing the collection using `iterator.remove()`
 - a `for` loop does not
- Iterators can be used for most collection types (e.g. `List`, `Set`, etc.)
- the specific class `ListIterator` can be used for bidirectional traversal of lists (forward and backward)

An example with element removal is given below:

```
[17]: Iterator<String> iterNames = names.iterator();

while (iterNames.hasNext()) {
    String c = iterNames.next();
```

```

        if (c.equals("Eric")) {
            iterNames.remove();
        }
    }

    System.out.println(names);

```

[Robert, Allison]

1.19.7 HashMap

HashMap is a JCF class that:

- stores data as key-value pairs
- maps keys to values
 - keys must be unique
- uses hashing for fast lookups
- has no guaranteed order

A HashMap is created using:

```
HashMap<KeyType, ValueType> hashMapName = new HashMap<KeyType, ValueType>();
```

A key-value pair can be added using:

```
hashMapName.put(key, value)
```

A value can be retrieved using:

```
hashMapName.get(key)
```

For example:

```

[18]: HashMap<String, String> specialties =
        new HashMap<String, String>();

specialties.put("Chase", "Intensivist");
specialties.put("Cameron", "Immunologist");
specialties.put("Foreman", "Neurologist");

System.out.println(specialties);
System.out.println(specialties.get("Foreman"));

```

```

{Foreman=Neurologist, Cameron=Immunologist, Chase=Intensivist}
Neurologist

```

1.19.8 HashSet

HashSet is a JCF class that:

- stores unique elements only
- is backed internally by a HashMap
- does not preserve insertion order (order is not guaranteed)

- uses hashing for fast lookup

A `HashSet` can be created using

```
HashSet<typeName> setName = new HashSet<typeName>();
```

Elements can be added using

```
setName.add(element);
```

To check if an element exists in the set, use:

```
setName.contains(element); // returns a Boolean
```

1.20 Stack and heap

In Java, there are two main memory regions used during program execution:

- Stack: stores method calls, parameters, and local variables
- Heap: stores objects and arrays created at runtime (dynamically allocated memory)

1.20.1 Stack

Whenever a function is called, the required memory for its local variables is already known beforehand:

- this memory gets allocated on the stack
- when a function calls another function internally, this second function's variables are 'stacked' on top
- once a function completes, its memory is automatically freed (removed from the top of the stack)
 - the programmer does not need to do anything

1.20.2 Heap

The heap stores memory allocated dynamically during runtime, such as objects and arrays created using `new`:

- references to heap objects may be stored in stack variables, fields of other objects, or static variables

Java automatically manages heap memory using a garbage collector.

- when an object is no longer reachable by any reference, it becomes eligible for garbage collection
- the garbage collector eventually reclaims its memory
- the exact time when this happens is not guaranteed

1.20.3 Objects and references in collections

Collections do not create copies of objects when they are added. Instead, they store references to the objects in the heap.

Therefore, if an object is modified through a reference outside the collection, the change is also visible when accessing the object through the collection, because both references point to the same

object.

For example:

```
[19]: class Person {
        String name;
        public Person(String pName) {
            this.name = pName;
        }
    }

    ArrayList<Person> people = new ArrayList<>();

    Person myPerson = new Person("Robert");

    people.add(myPerson);
    // the ArrayList stores a reference
    // to the Person object

    myPerson.name = "Eric";
    // modify the object through the
    // original reference

    System.out.println(people.get(0).name);
    // name has been changed, even when
    // accessed through ArrayList
```

Eric

1.20.4 Copy constructor

To modify an object without affecting other references to the same object, a separate copy of the object must be created and modified instead.

A common way to do this is with a copy constructor.

A copy constructor is an overloaded constructor that takes another object of the same class as a parameter and copies its data into the new object.

```
[20]: class Person {
        String name;
        public Person(String pName) {
            this.name = pName;
        }

        // copy constructor
        public Person(Person pPerson) {
            this.name = pPerson.name;
        }
    }
```

```

ArrayList<Person> people = new ArrayList<>();

Person myPerson = new Person("Robert");

people.add(new Person(myPerson));
// add a copy instead of the original object

myPerson.name = "Eric";

System.out.println(people.get(0).name);

```

Robert

1.21 Design patterns

Design patterns are reusable solutions to common problems in software design.

They:

- improve code readability, maintainability, and scalability
- promote best practices in software design
- provide proven solutions to common design problems
- help build flexible and maintainable systems

There are three main categories of design pattern:

- Creational patterns
 - deal with object creation mechanisms
 - focus on how objects are instantiated
- Structural patterns
 - deal with relationships between classes and objects
 - focus on how classes are composed or structured
- Behavioral patterns
 - deal with communication and interaction between objects
 - focus on how objects collaborate and share responsibilities

1.21.1 The Singleton pattern

The Singleton pattern is a creational pattern that ensures that a class has only one instance, and provides global (controlled) access to it.

It is useful for:

- logging
- timing methods or measuring code execution
- global configuration or constants
- multithreading (e.g. thread pool management)
- global synchronisation

```

class BasicSingleton {
    private static BasicSingleton instance;

```

```

    // holds the single instance of the class

    private BasicSingleton() {}
    // private constructor prevents external instantiation

    public static BasicSingleton getInstance() {
        if (instance == null) {
            instance = new BasicSingleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Singleton instance running");
    }
}

```

In a Singleton class:

- it holds a **static** instance of itself
 - this ensures the instance is shared globally
- the constructor is **private**
 - prevents other classes from creating new instances using **new**
- the instance is created only when first requested (lazy instantiation)

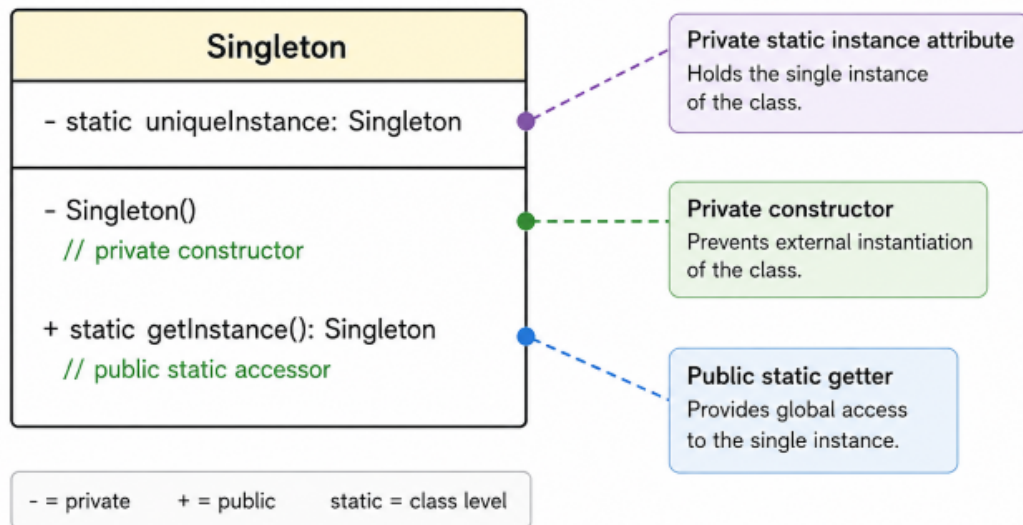
The Singleton can be accessed via the **static** method

`BasicSingleton.getInstance();`

Singleton in UML In UML, a Singleton class is typically shown with:

- a **private** constructor (to prevent external instantiation)
 - represented by - prefix
- a **private static** instance attribute (often named something like `uniqueInstance`)
 - represented by - **static** prefix
- a **public static** getter (e.g. `getInstance()` or `Instance()` method) that returns this single instance
 - represented by + **static** prefix

Singleton Design Pattern – UML Class Diagram



Caveats of Singletons Singletons:

- may introduce unnecessary or hidden dependencies between classes
- introduce global state into the program
 - global state can make code harder to reason about and test
 - can reduce modularity and flexibility
- can cause issues in multithreaded environments
 - e.g. race conditions if the Singleton is not implemented in a thread-safe way
- can make unit testing more difficult due to shared state persisting across tests

1.22 The Abstract Factory Pattern

This entire section (Abstract Factory + Factory) is written by ChatGPT. Je ne sais pas.

The Abstract Factory Pattern is a creational design pattern where:

- object creation is delegated to a factory object instead of creating objects directly
- a factory provides an interface for creating families of related objects (products)
- the client code does not need to know the concrete classes being instantiated

1.22.1 Key idea

The Abstract Factory creates **families of related objects** without exposing their concrete classes.

This makes the code more flexible and maintainable:

- object creation is encapsulated inside factory classes
- client code is decoupled from concrete implementations
- entire families of related objects can be swapped at runtime

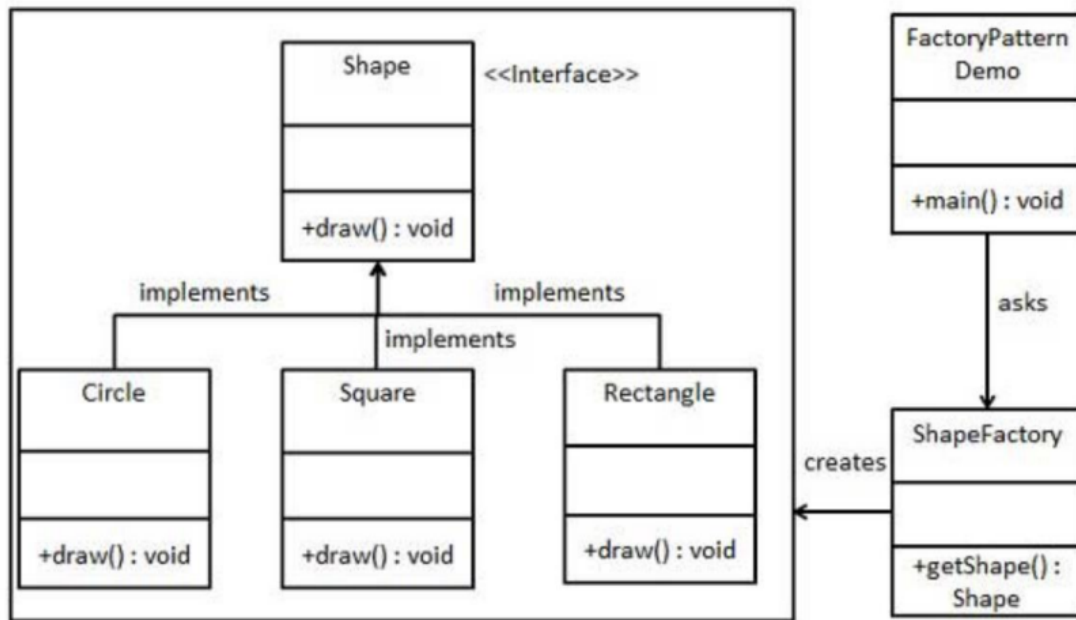
- new product families can be added with minimal changes

1.22.2 Example structure (conceptual)

- GUIFactory → creates UI components
- WindowsFactory → creates Windows-style components
- MacFactory → creates Mac-style components

Each factory creates a **matching set of products**.

1.22.3 UML



1.23 The Factory Method Pattern

The Factory Method Pattern is a creational design pattern where:

- object creation is delegated to a method instead of using constructors directly
- a superclass defines a factory method
- subclasses override this method to decide which concrete class to instantiate

1.23.1 Key idea

The Factory Method creates **one type of product**, but allows subclasses to decide which implementation is created.

1.23.2 Characteristics

- uses inheritance
- defines a factory method in a superclass
- subclasses override the factory method

- focuses on creating a single product hierarchy

1.23.3 Example (simple factory method style)

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing a circle.");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Drawing a square.");
    }
}

class ShapeFactory {
    public static Shape getShape(String type) {
        if (type.equals("CIRCLE")) return new Circle();
        if (type.equals("SQUARE")) return new Square();
        return null;
    }
}
```

Usage:

```
Shape shape = ShapeFactory.getShape("CIRCLE");
shape.draw();
```

1.23.4 Key difference

Factory Method Pattern

- focuses on one product type
- uses inheritance (subclasses override method)
- decides which single object to create

1.23.5 Abstract Factory Pattern

- focuses on families of related products
- uses composition (factory objects)
- creates multiple related objects together

1.24 The Observer Pattern

The Observer Pattern is a behavioural design pattern that defines a one-to-many dependency between a subject and multiple observers.

- the subject maintains a list of observers
- observers register (subscribe) with the subject
- when the subject's state changes, all registered observers are notified automatically
- each observer can then update itself in response to the change

1.24.1 Key idea

The Observer Pattern allows one object to broadcast changes to many dependent objects without needing to know their concrete classes.

This promotes:

- loose coupling between the subject and observers
- easy addition and removal of observers
- event-driven programming

1.24.2 Common uses

The Observer Pattern is commonly used when events need to be handled, for example:

- GUI: button clicks
- GUI: mouse movement
- receiving a new message or email
- listeners waiting for external events

1.24.3 Roles

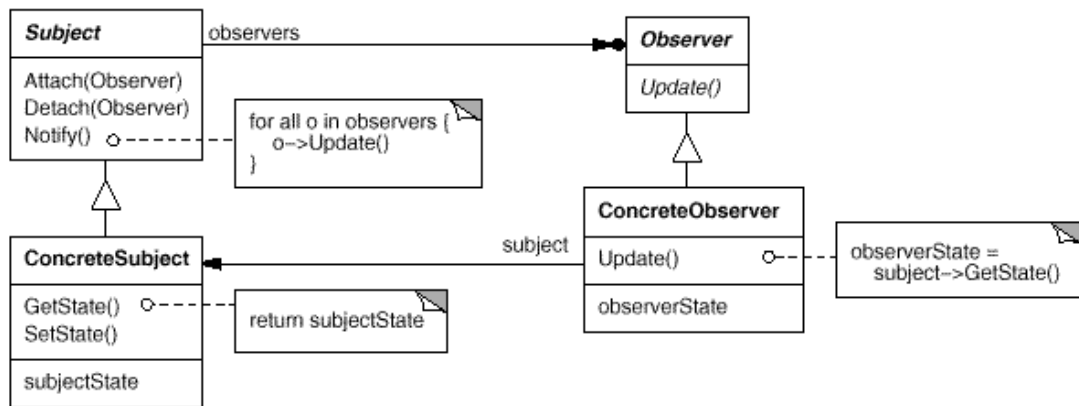
Subject (Observable):

- stores state
- maintains a list of observers
- notifies observers when its state changes

Observer:

- defines an `update()` method
- receives notifications from the subject

1.24.4 UML



[]: